

第4章 基于归纳的算法设计

4.1 引言

- 基本思想：从简单到复杂
- 途径：
 - 横向分解：将原问题分解为互不相同的子问题（模块化）
 - 纵向分解：将原问题分解为与原问题结构相似但规模更小的子问题，然后将子问题的结果归并（递归）
- 原理：解决小规模问题是可能的，且每个问题的解答都可由更小规模问题的解答构造出来
 - 归纳：问题规模由 $n - 1$ 到 n
 - 分治：问题规模由 $\frac{n}{k}$ 到 n
 - 动态规划：问题规模由 $< n$ 到 n
 - 增强归纳：由条件更多的问题求解原问题

4.2 多项式求和

- **问题**：给定 x 的值，求 $P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$ 的值
- **直接算法**： $P_n(x) = P_{n-1}(x) + a_n x^n$
 - 以乘法为基本的运算单元
$$T(n) = T(n-1) + n, T(n) = \frac{n(n+1)}{2} = \Theta(n^2)$$
- **改进（增强归纳假设）**：改为问题 $Q_n(x)$ ，既求 $P_n(x)$ ，也求 x^n
 - $Q_{n-1}(x) \rightarrow Q_n(x)$
$$x^{n-1} \rightarrow x^n, P_{n-1}(x) \rightarrow P_n(x)$$
需要 2 次乘法
 - $T(n) = T(n-1) + 2, T(n) = \Theta(n)$
- **进一步改进（改变归纳方向）**：改为问题 $R_n(x)$ ，其中 $R_0(x) = a_n, R_i(x) = x R_{i-1}(x) + a_{n-i}$
 - 每次迭代只需要 1 次乘法
 - $T(n) = T(n-1) + 1, T(n) = \Theta(n)$

4.3 最大导出子图

- **问题**：给定无向图 $G = (V, E)$ 及整数 k ，找到 G 中规模最大的导出子图 $H = (U, F)$ ， H 中所有顶点的度数 $\geq k$ ，或证明不存在 H
- **算法设计**：删除所有度数 $< k$ 的顶点，并一并删除新产生的度数 $< k$ 的顶点，而无需考虑删除的次序
- **算法正确性证明**：设算法对 $n - 1$ 个结点的图成立，则对于 n 个结点的图

- $n \leq k$ 所求导出子图不存在，算法成立
- $n = k + 1$ 图为完全图时为所求导出子图，否则所求导出子图不存在，算法成立
- $n > k + 1$ 时
 - 若所有结点度数 $\geq k$ ，则该图即为所求导出子图
 - 若 $\exists v, d(v) < k$ ，则 v 一定会被删除
删除 v 及相关的边后，由归纳假设知算法成立

4.4 一对一映射

- **问题：**给定集合 A 及函数 $f: A \rightarrow A$ ，寻找元素最多的 $S \subseteq A$ ，满足 $f: S \rightarrow S$ 且 f 为双射，求 S
- **归纳（减小问题规模）：**寻找 A 上未被其他元素映射到的 i （即 $\forall x \in A, f(x) \neq i$ ），将其删除，可使问题规模由 n 减小到 $n - 1$ ，同时将 i 删除后产生的未被其他元素映射到的元素也一并删除（可用队列实现）；若不存在这样的 i ，说明此时的集合即为所求的 S
算法的时间复杂度 $\Theta(n)$

4.5 社会名流问题

- **问题：** n 个人中，若 A 被其余 $n - 1$ 个人知道，但不知道其余 $n - 1$ 个人，则称 A 为社会名流；给定 n 个人相识关系的邻接矩阵，求这 n 个人中是否存在社会名流
- **归纳（减小问题规模）：**最多只有 1 个人是社会名流
当询问 A 是否知道 B 时，根据下述规则，可将问题规模由 n 减小到 $n - 1$
最后排除到只剩 1 个人，可以遍历邻接矩阵中对应的 1 行和 1 列以核验其身份
算法的时间复杂度 $\Theta(n)$
 - 若 A 不认识 B ，则 B 不是社会名流
 - 若 A 认识 B ，则 A 不是社会名流

4.6 轮廓线问题

- **问题：**给出 n 个建筑的二维轮廓，求城市的轮廓线，其中建筑的 ADT 为 $\langle L, H, R \rangle$ ， L 为左基线， H 为高度， R 为右基线
- **直接方法：**每添加一栋建筑更新一次轮廓线
 $T(n) = T(n - 1) + \Theta(n)$ $T(n) = \Theta(n^2)$
- **优化（分治算法）：**将 2 个轮廓线合并依然是 $\Theta(n)$ 时间
 $T(n) = T(\frac{n}{2}) + \Theta(n)$ $T(n) = \Theta(n \log n)$
- 分治算法框架：

```

solve_or_divide(P) begin
  if (P.size() <= n0) solve(P)
  divide(P) // divide P into P_1, P_2, P_3, ..., P_k
  for (i = 1 to k) solve_or_divide(P_i)
  merge(result[1..k])
  return result
end solve_or_divide

```

- 分治算法流程：
 - 分：将原问题分解为 k 个子问题，每个问题规模为 $O(n)$
 - 治：若问题规模大于阈值 n_0 ，则进一步分解；否则求解该问题
 - 合：整合 k 个子问题的解，形成原问题的解
- 复杂度分析：

$$T(n) = \text{div}(n) + \sum_{k=1}^n T(k) + \text{merge}(n, k)$$

4.7 计算平衡因子

- **问题**：给定二叉树，计算内部结点和根节点的平衡因子
- **归纳**：由于平衡因子不依赖于节点的平衡因子，而只依赖于树的高度
因此增强归纳假设为求出平衡因子，同时给出子树高度，便于求父结点的平衡因子

4.8 最大连续子列和

- **问题**： $x[1..n]$ 为实序列，求 $x[i..j]$ 使得其和为原序列的所有连续子列和中最大
- **归纳**：假设已找到 $x[1..n-1]$ 的最大连续子列和
 - 若最大和的连续子列为 $x[i..n-1]$ ，只需要考虑 x_n 是否大于 0 即可
若 $x_n > 0$ ，则将 x_n 加入所求子列，否则不加入
 - 若最大和的连续子列为 $x[i..j] (j < n-1)$ ，需要比较 $x[i..j]$ 的和与 $x[1..n-1]$ 的最大后缀子列和加上 $x[n]$ 的和
因此将归纳假设增强为：既求出最大子列和，也求出最大后缀子列和
维护最大后缀子列和的方法为：若 $x[n]$ 加入使得最大后缀子列和变为负数，则将最大后缀子列和重置为 0 即可；否则最大后缀子列和不变
算法时间复杂度 $\Theta(n)$ （只需要扫描序列 1 遍）

4.9 动态规划：背包问题

- **问题**：背包中有 n 个物品，价值分别为 $k[1..n]$ ，求物品组合使得其价值之和恰好为 k ，或说明不存在这样的物品组合

- **直接归纳假设**：假设已知 $k[1..n-1]$ 中是否存在这样的物品组合
但若答案为不存在，则归纳假设不能用于求解 $k[1..n]$
- **增强归纳假设**：假设可以求解 $k[1..n-1]$ 中是否存在这样的物品组合
 $T(n) = 2T(n-1)$ $T(n) = \Theta(2^n)$ 复杂度很高
- **强归纳法（动态规划）**：不仅利用 $k[1..n-1]$ 的解，而是利用所有小规模问题的解
已知 $k[1..i-1]$ 中是否存在价值为 $[1..k-1]$ 的物品组合，记为 $P[i-1, j]$ ($j = 1, 2, 3, \dots, n-1$)
则当 $k[n]$ 加入时有

$$P[i, j] = \text{true} \Leftrightarrow P[i-1, j] = \text{true} \vee P[i-1, j-k[i]] = \text{true}$$

- **动态规划求解流程**：设立二维数组 $P[0..n, 0..k]$ 保存小规模问题的解
初始化 $P[0, 0] = \text{true}, P[0, 1..k] = \text{false}$
然后每次加入 $k[i]$ 时，依次求解 $P[i, 0..k]$
最后取 $P[n, k]$ 为问题解
- **时间复杂度**：由于求解二维数组的每个元素均为 $\Theta(1)$ 时间
所以算法的时间复杂度为 $\Theta(nk)$

4.10 总结

- 常见错误：
 - 不添加递归终止条件导致无限递归
 - 从 n 扩展到 $n+1$ 时忽略 $[n+1]$ 元素的一般性
- 核心思想：小规模问题与大规模问题的联系
 - 直接方法：去掉某些元素
 - 间接方法：分解为多个小规模问题
- 分治算法与动态规划的差异：
 - 分治算法：递归思想的应用
 - 动态规划：循环迭代思想的应用